Consider the mapping function with identified points P1 and P2 shown in Figure 2. The projection line is a line of unit slope which passes through P1. The second identified point P2 lies below the projection line. Assuming only first order fragmentation, it is impossible to interpolate a valid mapping function between P1 and P2, because the function is not invertible1. This property of mapping functions can be used in many cases to immediately spot situations where first order frag mentation is impossible, and an exhaustive search is re quired. 3. THEJPEGFILEFORMAT JPEG is the most common file format for storage and distribution of photographic imagery. This standard is de fined in detail in [9], but here we discuss those aspects of the format which are directly relevant to the construction of carvers. A JPEG file is divided into a series of markers represent ing different sections within the file. Although the markers do contain length parameters for the size of the headers, compressed data appears between the headers and has no length parameter. In particular the SOS (Start of Scan) marker header is followed by the ECS (Entropy Coded Seg ment) which does not have its length specified in advance. In addition, the ECS section is typically the largest section of the file, representing the majority of data in the JPEG file. Unlike the examples presented by [6] of the PDF and ZIP file formats, the JPEG file format is less structured. It is impossible to know in advance the file offset of most markers. This is mainly due to the fact that some sections are un sized. This makes it difficult to find a sufficient number of positively identified points. Unfortunately this implies that JPEG carvers usually need to resort to exhaustive searches. 3.1 TheJPEGDiscriminator 1Mapping functions must be invertible because there is a 1 1 relation between file offsets and image offsets. The above example has regions where image data is used twice in the file Figure 3: An overview of libjpeg architecture As mentioned in 2.1.2, a good discriminator will provide a good fix on the location of the discontinuities within the file. For the JPEG discriminator, we need to identify the sector offset of the discontinuity as accurately as possible. Our discriminator uses the libjpeg decoder[11]. This li brary is extremely well designed and supports a number of operating modes which are useful in different situations. The strength of the library is that its possible to replace parts of the decoder to achieve different goals. Figure 3 illustrates how libjpeg is used: • Adecoder loop retrieves each scan line using the jpeg read scanlines function. • The main JPEG decoder then relies on its subsystems to decode the required scan lines and return them. Only complete scan lines are returned. • When the decoder requires more data it calls fill input buffer from the source manager. This al lows the source manager to provide compressed data in small chunks. • The decoder uses the memory manager to allocate memory for its internal operation. • If an error is detected by the decoder, the decoder calls the error manager to display the message using its output message function. In our case we want to detect errors as soon as they occur. We therefore create our own error manager to monitor the error count from the decoder. Our source manager then provides the decoder with one sector at the time and watches the error count in order to detect the exact sector which causes an error. Unfortunately the jpeg decoder itself is often unable

to detect errors soon enough after they occur. This makes the detection of bad sectors unreliable. Figure 4 illustrates an example of an image containing a major corruption but without causing libjpeg errors. Al though the corruption can easily be detected visually, the de coder is unable to find errors within the compressed stream. Oncloser examination of the corrupted region, as illustrated 1500 1000 Integral Value Figure 4: An example of an image corruption which is not detectable by libjpegs error detection. This image was taken from the 2007 DFRWS header off set 90377 sectors. The inset illustrates the boundary between the scan lines which are correct and those which are corrupted. 500 0 0 100 200 300 400 500 600 Scan Line 700 800 900 1000 1100 1200 Figure 5: A graph of the value of the estimate Eq. 1 calculated for each scan line of the image shown in Figure 4. As can be seen at scan line 594 the integral rises dramatically with severe oscillations from then on. by the inset, we can see that there is a clear edge between the two regions based on the different textures between the corrupted region and the correctly decoded region. Our carver uses an edge detection technique to estimate the error levels in addition to libjpegs internal error detec tion mechanisms. This allows us to be more accurate with detecting possible errors. The presence of an edge is esti mated by calculating the estimate: width X E(y) = x=0 ¸ ¸ ¸ ¸fx,y − fx,y+1 + fx,y−1 ¸ ¸ ¸ 2 ¸ (1) Where E(y) is the estimate of scan line y, and fx,y is the component value at scan line y and pixel x. The estimate is thus merely an integral of the difference between the actual component value of a line and the expected value derived by averaging the line above and below it. For JPEG pho tographic images component values tend to vary smoothly at the pixel level. In deriving estimate we therefore apply a linear approximation to the vertical component of the com ponent values. An example of this estimate for all scan lines is shown in Figure 5. Sometimes the bad sector will cause a corruption in the middle of a scan line. This makes it difficult to detect be cause the decoder will typically consume a number of sectors in order to complete the line before returning the full scan line to us for edge detection. Our carver solves this problem by inserting an end marker (Hex code 0xFFD9) code imme diately after the sector of interest. This causes the decoder to flush all compressed data and return a partial line. We are then able to determine how many horizontal pixels were decoded and restrict our edge detection to the difference be tween the decoded frame resulting from decoding subsequent sectors. Figure 6: An example of an image corruption which can be revealed by decoding sectors one at the time. This image (of a dog) was taken from the 2007 DFRWS at header offset 87716 sectors. The top part is shown after decoding 243 sectors, while the bottom part was after decoding 244 sectors. The black bands illustrate the region over which the Es timate (Eq. 1) was taken. This region is restricted to the boundary between the smaller image and the image obtained by decoding more data. The estimate (Eq. 1) is then calculated for the difference between the image resulting from decoding the previous sec tor and this sector. Figure 6 illustrates this process. The top part of this figure shows the image resulting from decoding only 243 sectors of the image (and artificially inserting an End Of Image (EOI)

marker). The bottom part of this image shows the decoded result of decoding 244 sectors. The edge detection algorithm is then applied to the difference between the two frames and indicates an error. 3.2 Themapping function generator The mapping function generator is a component of the carver which is responsible for choosing possible mapping functions to be tested by the discriminator. In our case the generator is focused on resolving each discontinuity sepa rately. The discriminator is asked to evaluate the validity of the mapping function only up to a point shortly after the discontinuity. This allows each discontinuity to be resolved Figure 7: Overview of Carving algorithms in turn until the complete image is recovered. 3.3 Carving Algorithm The overall algorithm is illustrated in Figure 7: • The image is preprocessed to locate JPEG headers. This positively identifies a single point on the mapping function. • The mapping function is decoded a sector at the time to establish the point at which errors begin to occur. This provides a narrow range on the file offset where a discontinuity may exist. • A function generator is used to choose possible sec tors to append at the identified discontinuity and the discriminator is asked to assess the resultant image if decoded a few sectors past the discontinuity. • When a sector was found which does not result in er rors, the algorithm goes back to identifying the next discontinuity. 3.4 Fragmentation model Fragmentation occurs in hard disks as a result of filesys tem allocation strategy. These strategies are typically de signed to optimise some characteristic of the filesystem (e.g. faster access speed, better storage efficiency etc) and are not deliberately designed to make recovery difficult. It is often possible to make assumptions about the type of fragmen tation present based on information about the file system which was previously on the disk and its typical allocation strategies. A fragmentation model is a set of assumptions derived from an observation of how fragmentation occurs in prac tice. This may depend on filesystem characteristics or other simplifying assumptions. The model can be applied to the generator by reducing the number of mapping functions which need to be examined, or having the mapping functions tested in likely order of occurrence. For example the DFRWS challenge [3] states the following assumptions: • Files begin on sector boundaries. • Fragmentation can only occur on sector boundaries. • Sectors are 512 bytes in size. Figure 8: Decoder suspension techniques. Top: Memory based suspension. Bottom: Process based suspension. We also assume that fragments are more likely to be lo calised. That is, to say that sectors close to the file header are more likely to match than sectors further away. This assumption is common in real filesystems which attempt to keep file fragments physically close on the disk. For example [7] shows that in the wild most bifragmented files were sep arated by a reasonably small number of sectors. The same reference also shows that modern filesystems such as Ext2 and NTFScontain files which are fragmented on a filesystem blocksize of 4kb even larger. This allows for more accurate determination of the exact block of corruption by the dis criminator because there is more data within the incorrect block to cause more of the image to be decoded incorrectly- thereby increasing the value of the estimate. 4. OPTIMIZATIONS Although

the carver presented in this work is very effective in determining the correct mapping function, it still needs to perform an exhaustive search. For each sector tested, the decoder is used to decode the full image from the start until a particular point in the image (a little after the discontinuity identified). This is very inefficient because for subsequent decoding operations, most of the image is exactly the same, differing only in its final sectors. Ideally the decoder could be suspended at the point just prior to the position where the discontinuity can be found. The decoder can then be used to decode the region in ques tion and rewind back to the suspension point without need ing to re-decode the file from the start. Unfortunately, lib jpeg is not designed in this way and does not provide means to suspend the decoder. We devise two novel approaches of suspending the decoder without needing to modify the source code of the decoder itself. The following two techniques are illustrated in Figure 8. 4.1 Memorysuspension One can think of the decoder as a finite state machine. The future evolution of the decoder is fully determined by its current state and future input. Since libjpeg is a re entrant library, it follows that no state information can be stored within static library variables. All state information decoder. is stored by memory allocated to the specific instance of the decoder. As shown in Figure 3, libjpeg relies on a memory manager subsystem to allocate internal memory through a specialised allocator. We introduce a special memory manager which allocates memory from a single primary pool (Figure 8). The pool can be copied on demand to a shadow pool in order to form a snapshot of the decoder's memory without needing to understand the specifics of how the decoder op erates. This memory snapshot in addition to the file offsets of any file descriptors embodies the full decoder state. This can be used to restore the decoder to a known state by restoring the pool from its snapshot in the shadow pool. In addition to the memory, we need to also rewind file de scriptors to their saved offsets. The overall result of this is that a snapshot is created just before the identified discontinuity, and then this snapshot is resumed with each new sector tested. The decoder then decodes just the new region of the image from the point of the snapshot on, rather than decoding the entire image from the start. Because using the memory suspension technique we have a well defined concept of the decoder state (namely the pri mary allocation pool contents and file descriptor offsets), this state can easily be shared by different machines or pro cesses on the same machine. This approach allows us to spread the testing loads between many CPUs and machines in a cluster, thereby taking advantage of any parallelism in herent with the carving process. 4.2 Forking/Process suspension technique The forking/process suspension technique is much sim pler to implement than the memory suspension technique described above. The basic technique relies on the decoder process forking at the suspension point, sending its child process to continue decoding the test function. The parent then waits for the child to exit with an exit code which in forms the parent about the result of the test (A pass/fail exit code). The technique works with any decoder which decodes the file linearly (e.g. MPEG, Zlib, etc), without knowledge of the internal decoder state. The process suspension

technique allows the decoder pro cess to fork several times simultaneously, and thereby have several child processes test a number of functions simultane ously. This approach takes advantage of multiple processors on the system. Unlike the memory suspension technique, however, its difficult to share decoder states between differ ent systems since the decoder state is effectively represented by the process's memory image and can not be easily ex tracted. 5. CONCLUSIONS Data carving is a very promising forensic technology. This paper examined the JPEG file format and the development of efficient and automated carvers for it. A discriminator was presented which used edge detection and texture mapping operations to determined errors in the decoded image. The discriminator was based on libjpeg and operates by examin ing errors in the decoded image as each sector is processed. Finally an optimization was presented which relies on being able to store and resume the state of the decoder at arbi trary points without knowledge of the exact operation of the Although this paper concentrated on the JPEG file for mat, the techniques described can be applied equally well to all image formats. In particular the detection of discontinu ities based on image properties such as textures and edges can be applied to any image formats (or indeed video for mats). 6. REFERENCES [1] B. Carrier. File System Forensic Analysis. Addison Wesley Professional, March 2005. [2] B. Carrier, E. Casey, and W. Venema. DFRWS 2006 forensics challenge, URL http://dfrws.org/2006/challenge/ 2006. [3] B. Carrier, E. Casey, and W. Venema. DFRWS 2007 forensics challenge, URL http://dfrws.org/2007/challenge/ July 9 2007. [4] M. Cohen. Advanced carving- DFRWS submission, URL http://sandbox.dfrws.org/2007/cohen/ 2007. [5] M. Cohen and D. Collett. Python Forensic Log Analysis GUI (PyFlag), URL http://www.pyflag.net/ 2005. [6] M. I. Cohen. Advanced carving techniques. Digital Investigation, 2007. In Press. [7] S. L. Gafinkel. Carving contiguous and fragmented files with fast object validation. Digital Investigation, 4(Supplement 1):2–12, September 2007. [8] C. Grenier. Photorec, URL http://www.cgsecurity.org/wiki/PhotoRec 2007. [9] Joint Photographic Experts Group. JPEG standard (JPEG ISO/IEC 10918-1 ITU-T Recommendation T.81), URL http://www.w3.org/Graphics/JPEG/itu-t81